

OWASP Top 10 — Evidencias de Laboratorio

Explotación de vulnerabilidades en bWAPP (BeeBox)

Gonzalo Rodríguez de Mello

4Geeks Academy — Especialización en Ciberseguridad

23 de junio de 2026

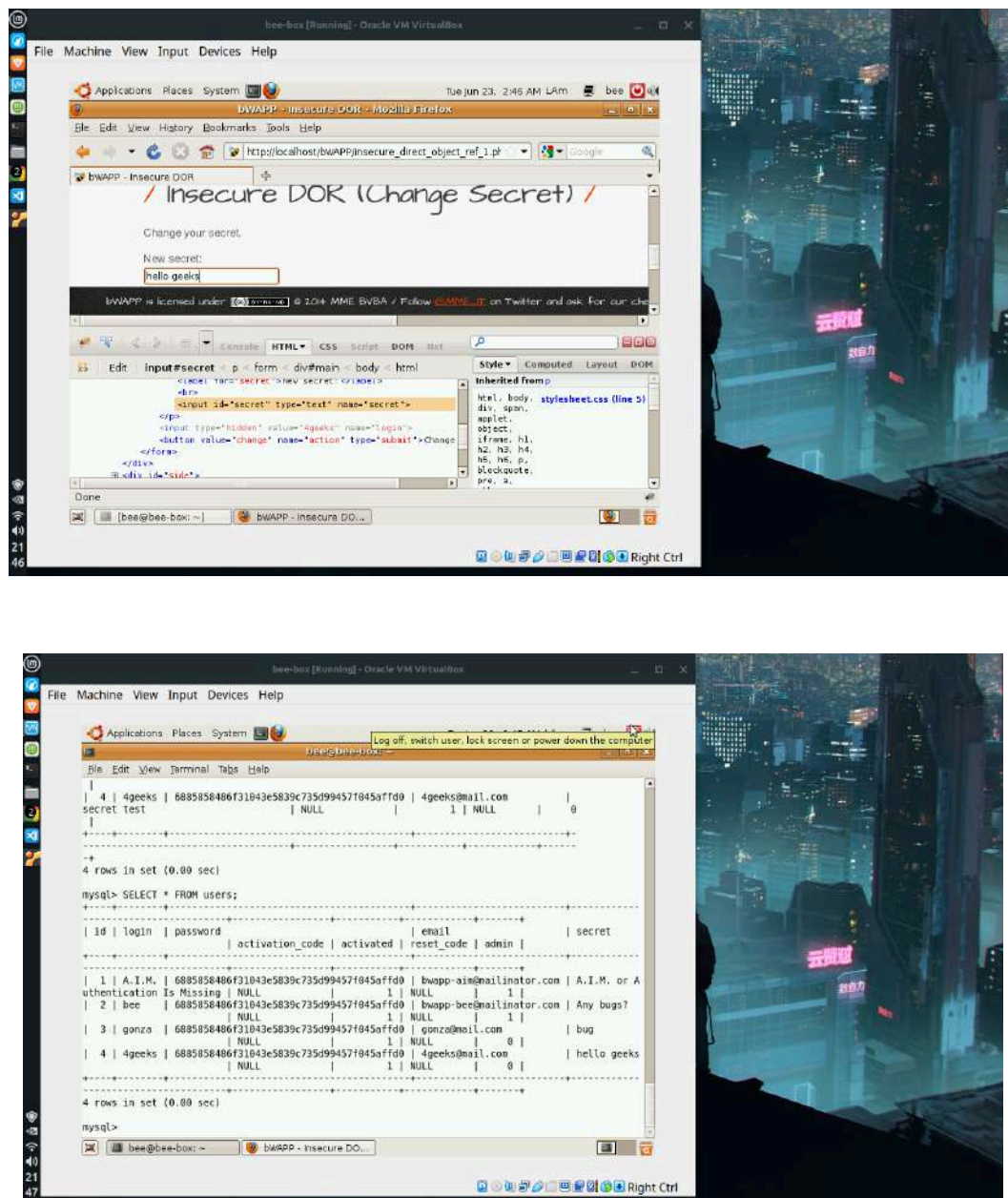
Introducción

Este documento reúne la evidencia de la resolución de los diez ejercicios del proyecto OWASP Top 10 de 4Geeks Academy, realizados sobre la aplicación vulnerable bWAPP alojada en la máquina virtual BeeBox. Cada ejercicio corresponde a una categoría del OWASP Top 10 y documenta la explotación de una vulnerabilidad concreta. Para cada uno se incluye una descripción del procedimiento realizado y las capturas de pantalla que evidencian el resultado obtenido.

1. Broken Access Control — Insecure DOR (Change Secret)

Categoría OWASP: A01 Broken Access Control.

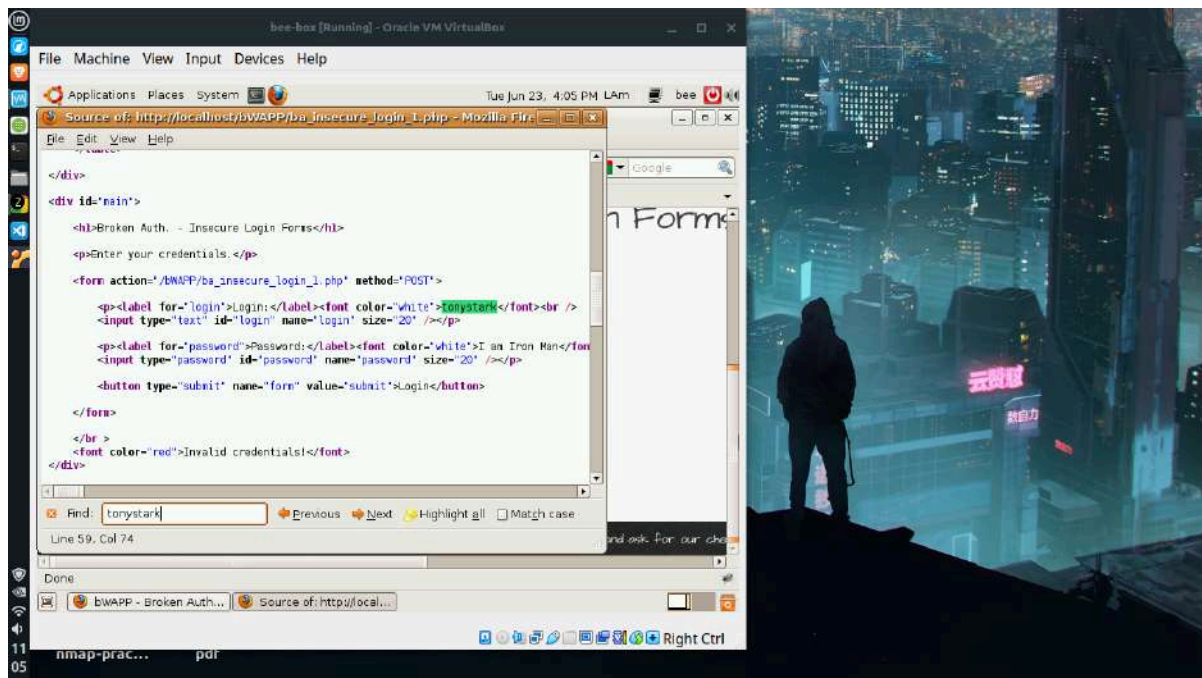
El ejercicio explota una referencia directa e insegura a objetos (IDOR). El formulario de cambio de secret de bWAPP incluye un campo oculto que indica a qué usuario pertenece la operación. El servidor confía ciegamente en ese valor sin verificar que el usuario autenticado sea el dueño de la cuenta. Manipulando el campo oculto desde las herramientas de desarrollador del navegador, se cambió su valor de “bee” a “geeks”, de modo que al enviar el formulario se modificó el secret de otro usuario. La verificación posterior en MySQL confirmó que el registro del usuario geeks quedó alterado sin haber iniciado sesión como ese usuario.

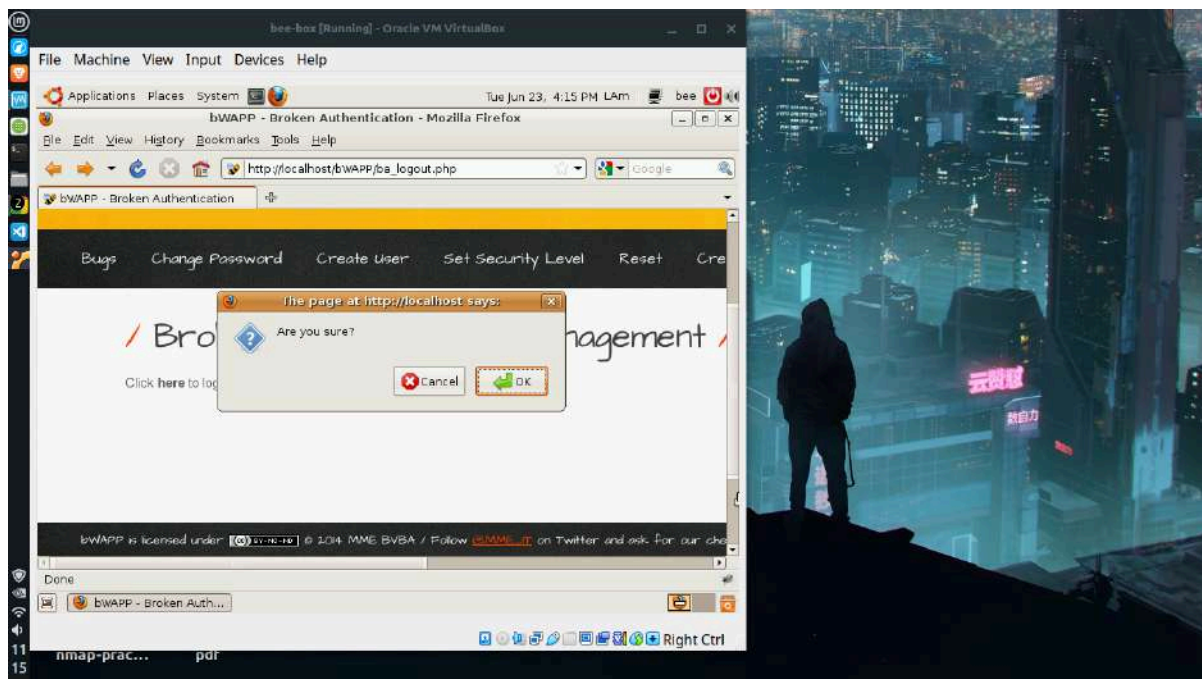
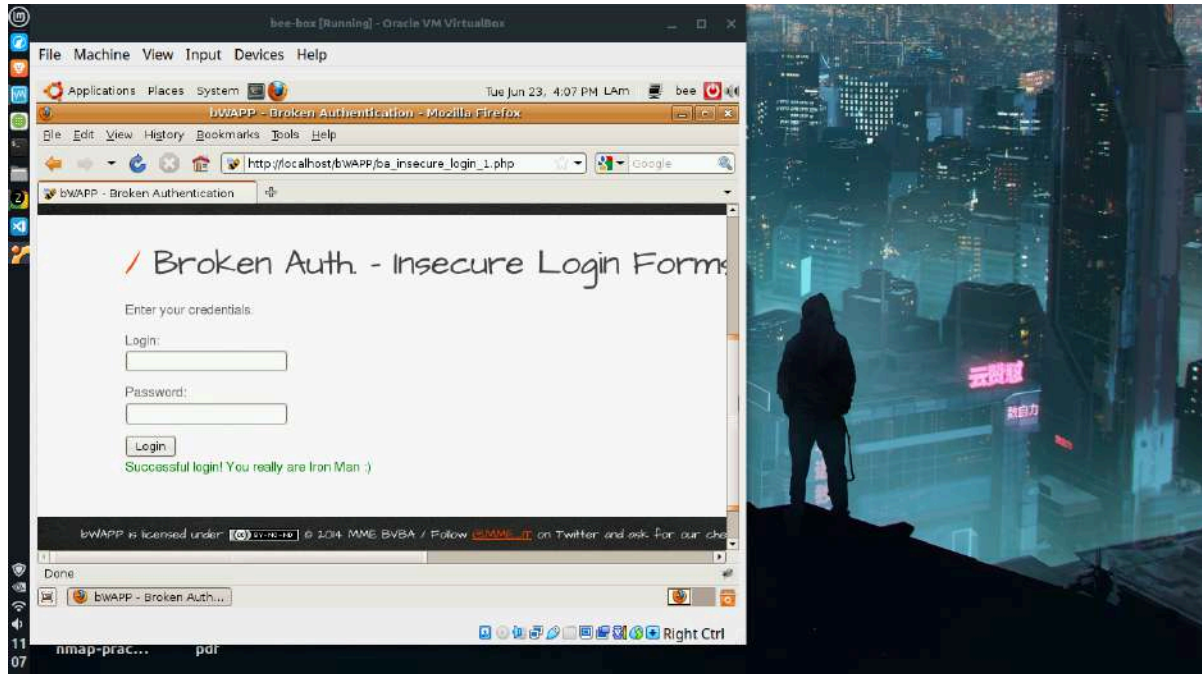


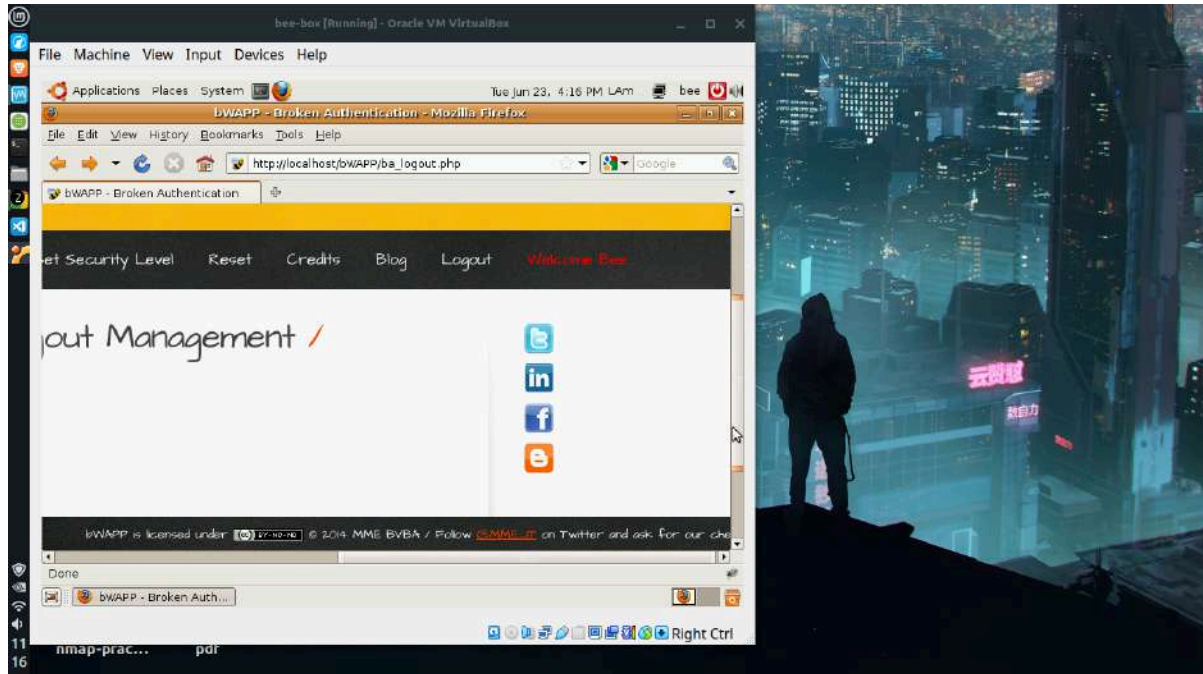
2. Identification & Authentication Failures — Broken Authentication

Categoría OWASP: A07 Identification and Authentication Failures.

El ejercicio abarca dos fallos de autenticación. En el primero, las credenciales del usuario `tonystark` se encuentran incrustadas en texto plano dentro del código fuente HTML de la página de login, visibles con la opción “Ver código fuente” del navegador. Usando esas credenciales se obtuvo acceso al sistema. En el segundo, se demostró una gestión de sesión defectuosa: tras hacer clic en “Logout” el sistema redirige al login aparentando haber cerrado sesión, pero al presionar el botón “atrás” del navegador se regresa a la página autenticada con acceso completo, evidenciando que la sesión nunca se invalidó en el servidor.



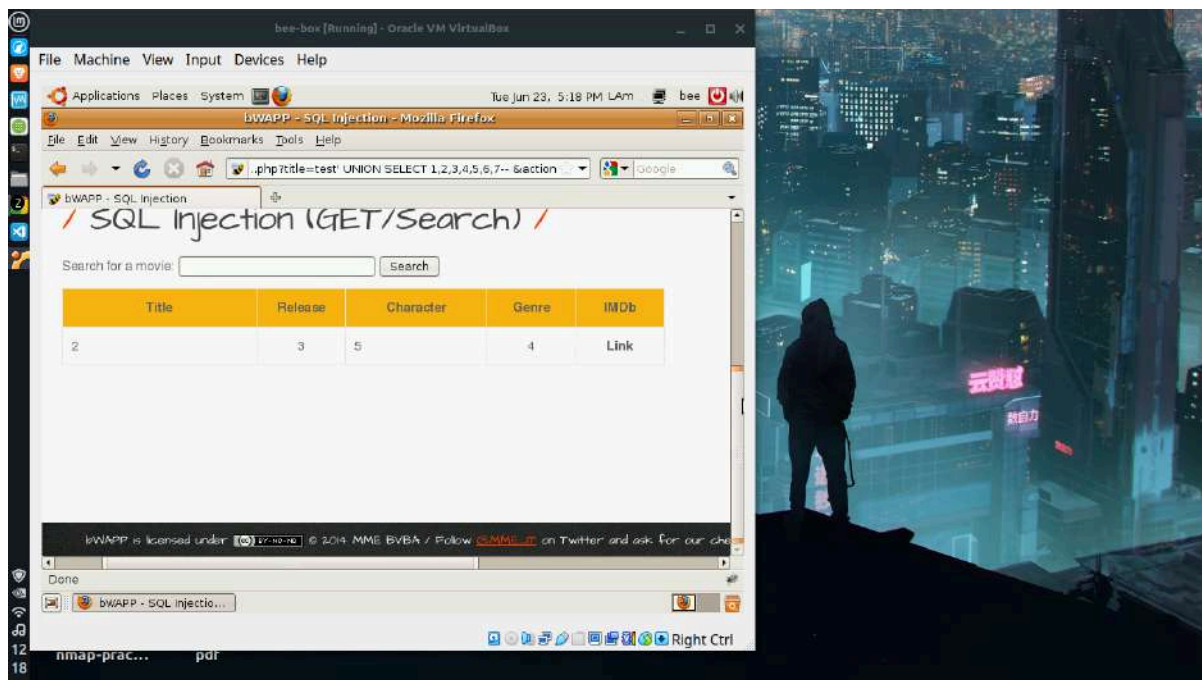


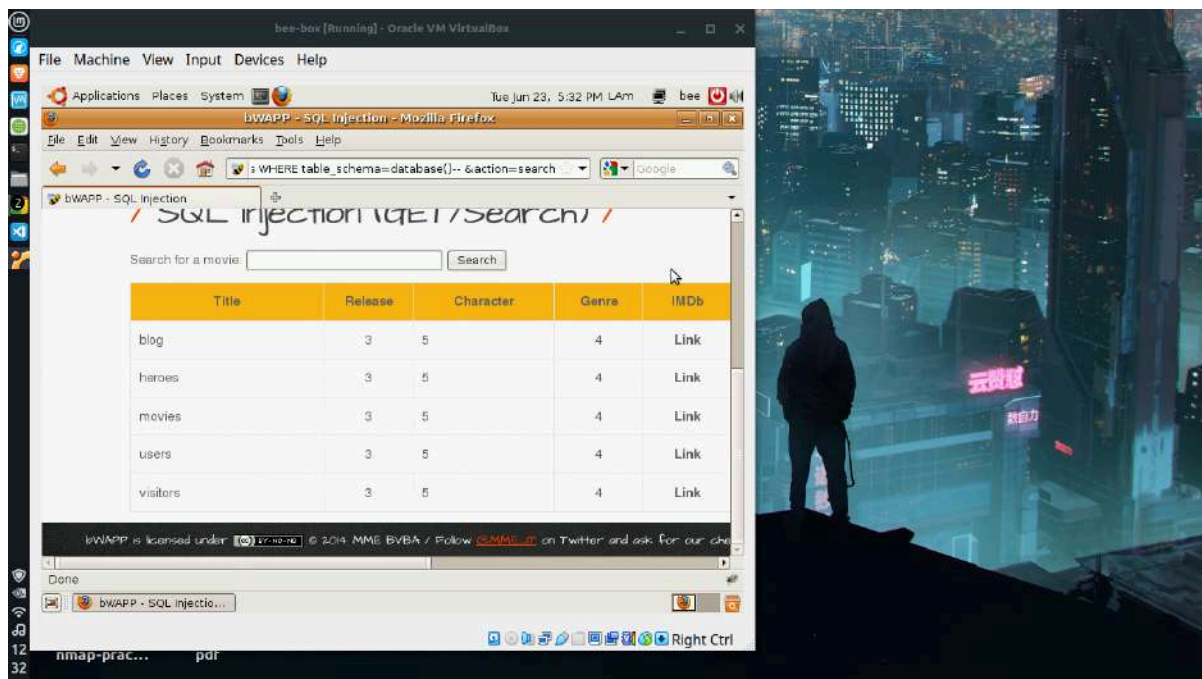
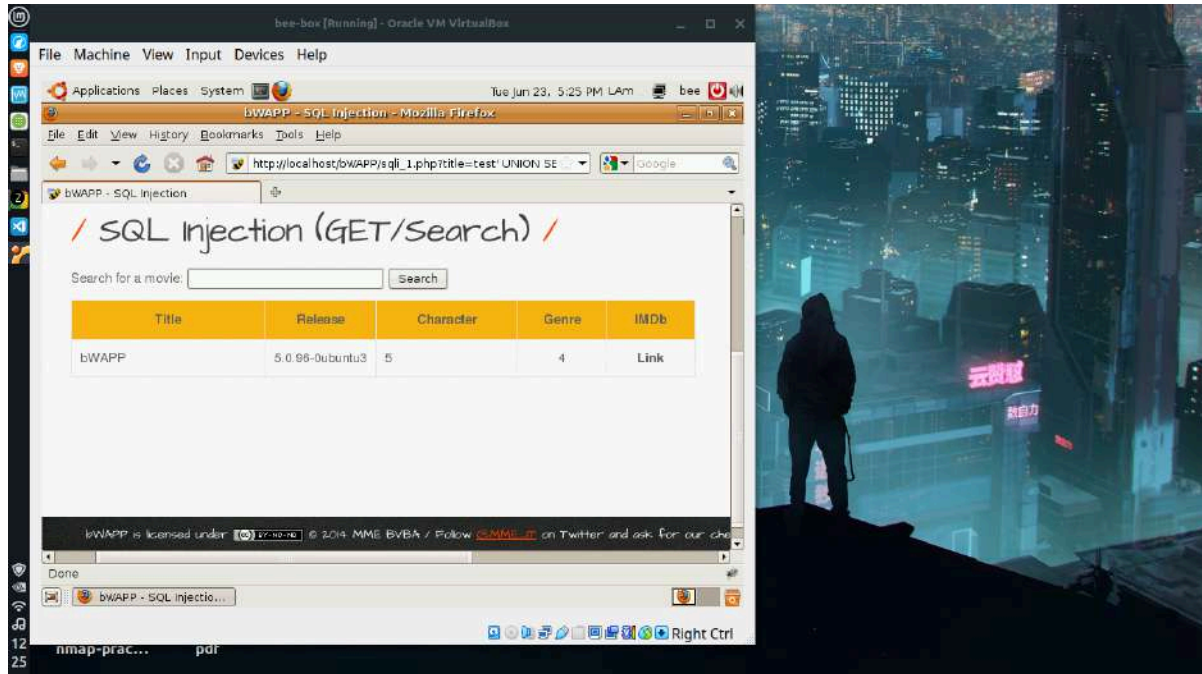


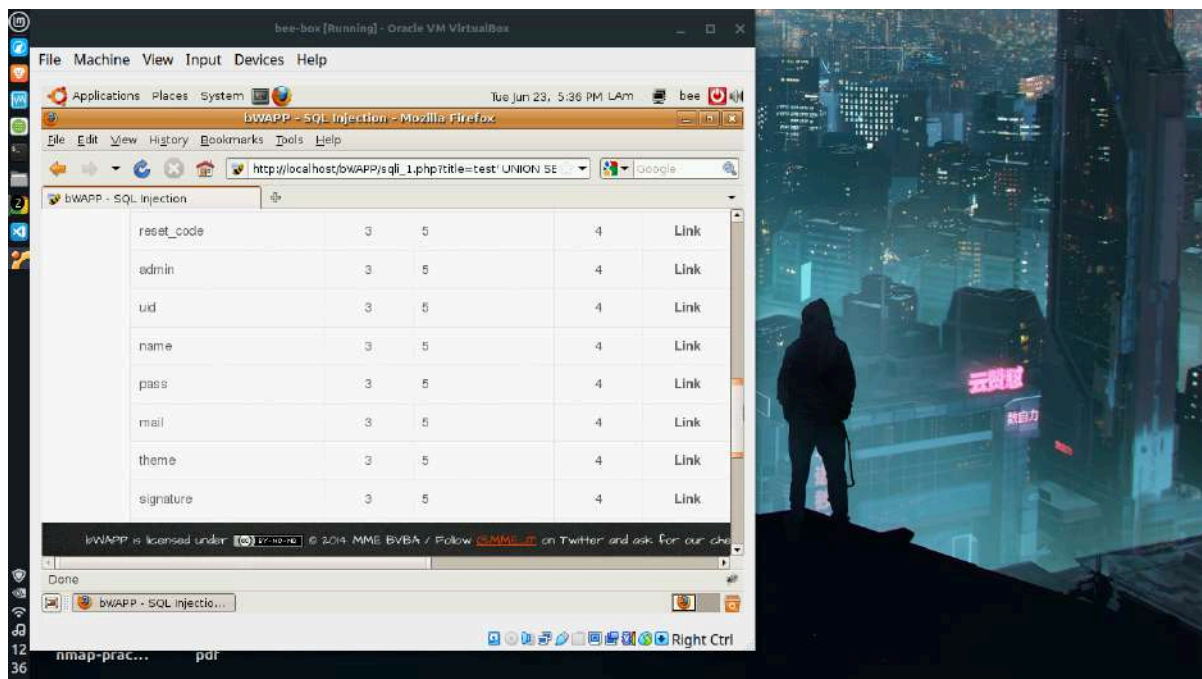
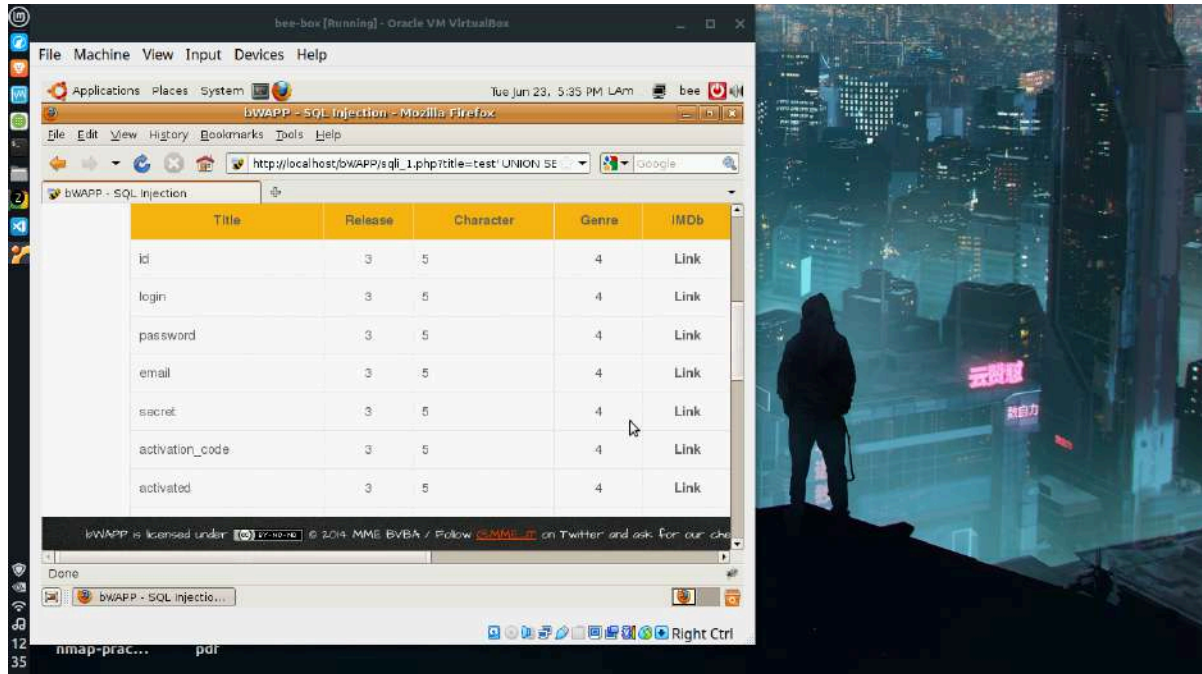
3. Injection — SQL Injection (GET/Search)

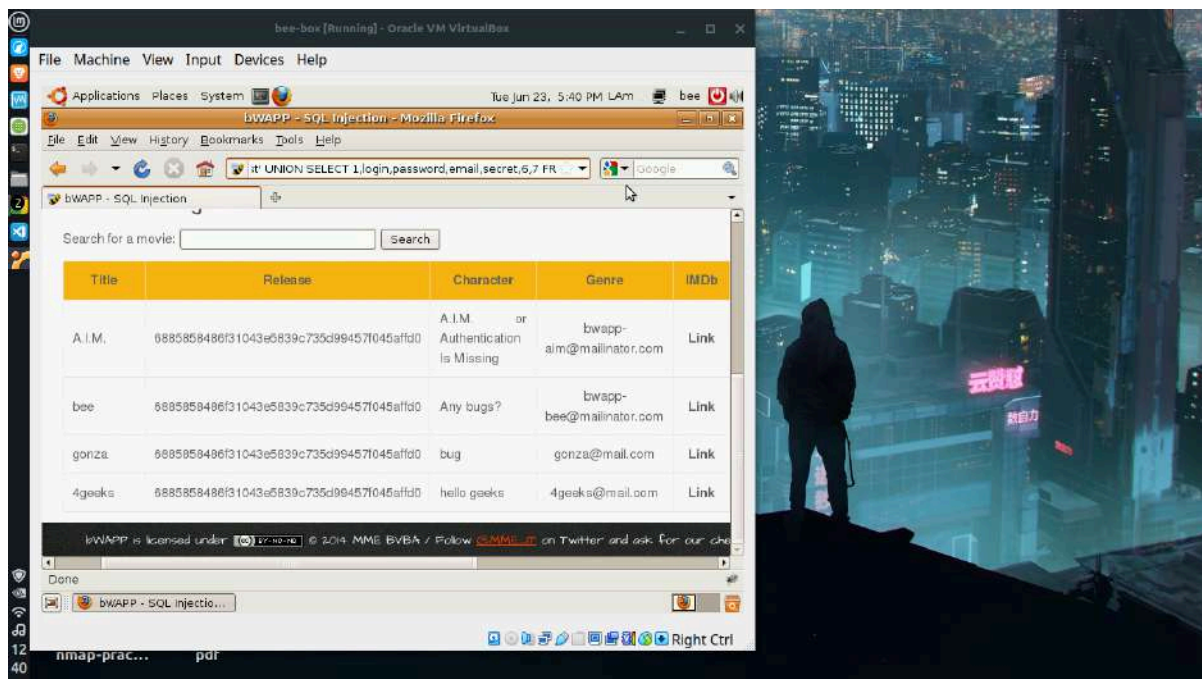
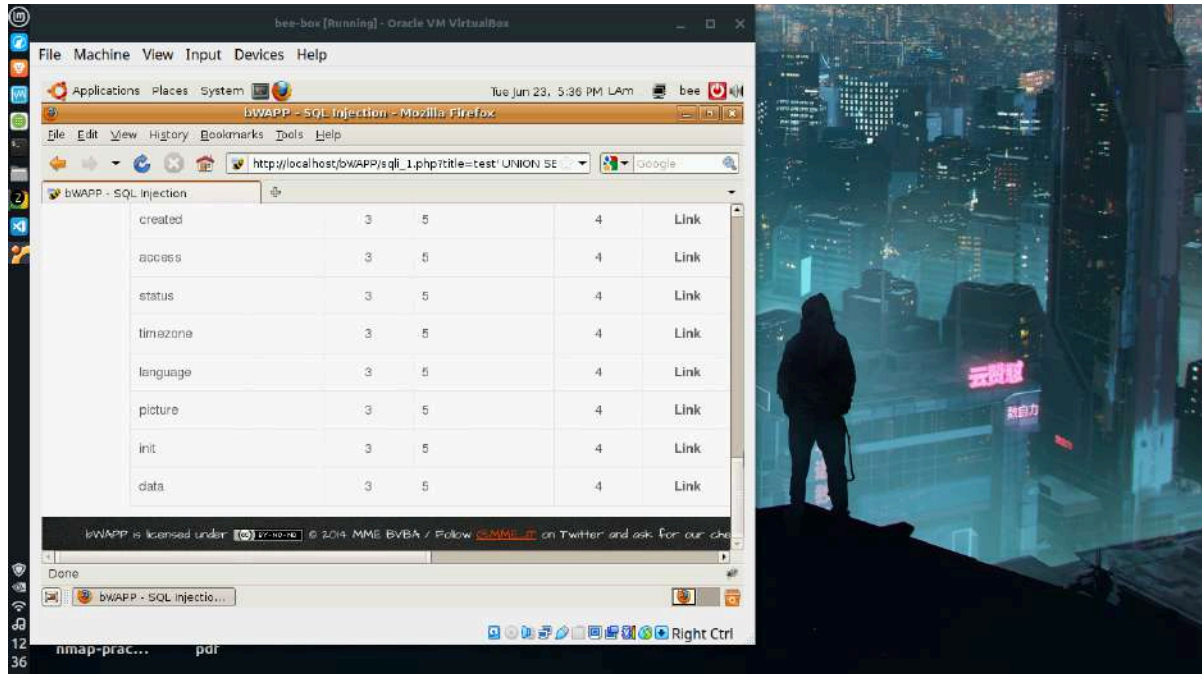
Categoría OWASP: A03 Injection.

El ejercicio explota una inyección SQL del tipo UNION-based sobre el formulario de búsqueda de películas. Primero se determinó la cantidad de columnas de la consulta original mediante la cláusula ORDER BY, estableciendo que la consulta devuelve siete columnas. A continuación, con inyecciones UNION SELECT se extrajo progresivamente el nombre de la base de datos (bWAPP), la versión del servidor (5.0.96-0ubuntu3), el usuario de conexión (root@localhost), el listado de tablas de la base, las columnas de la tabla users y, finalmente, las credenciales completas de todos los usuarios. La conexión con privilegios de root agrava el impacto, ya que viola el principio de mínimo privilegio.





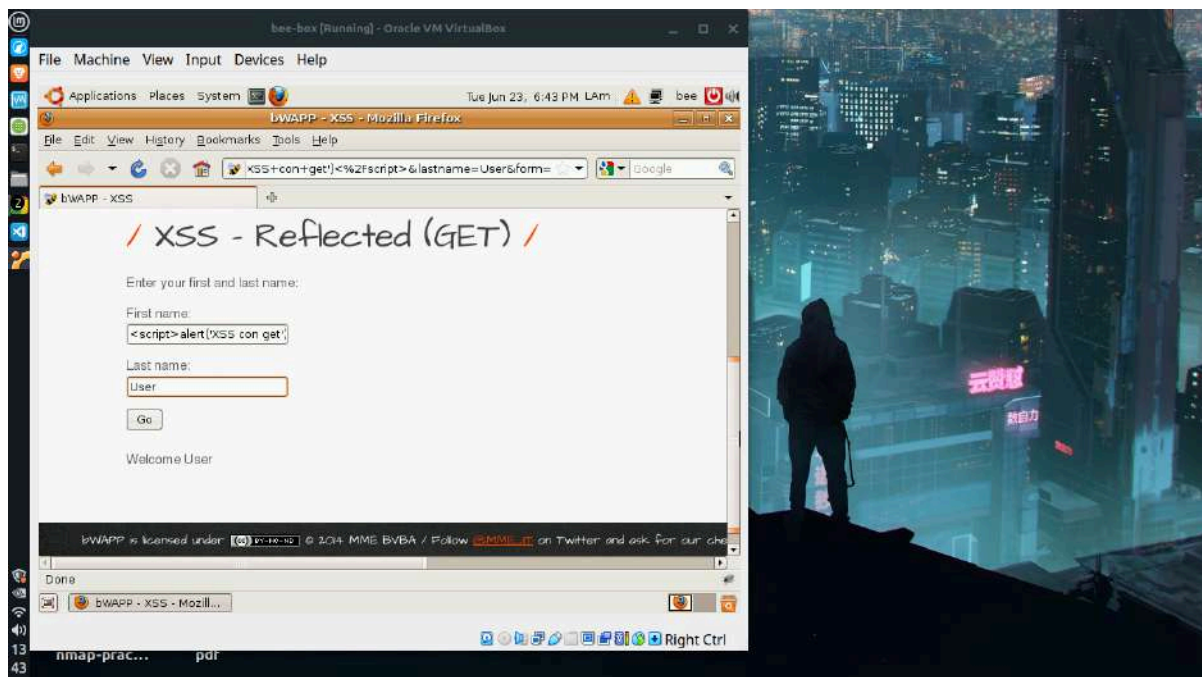


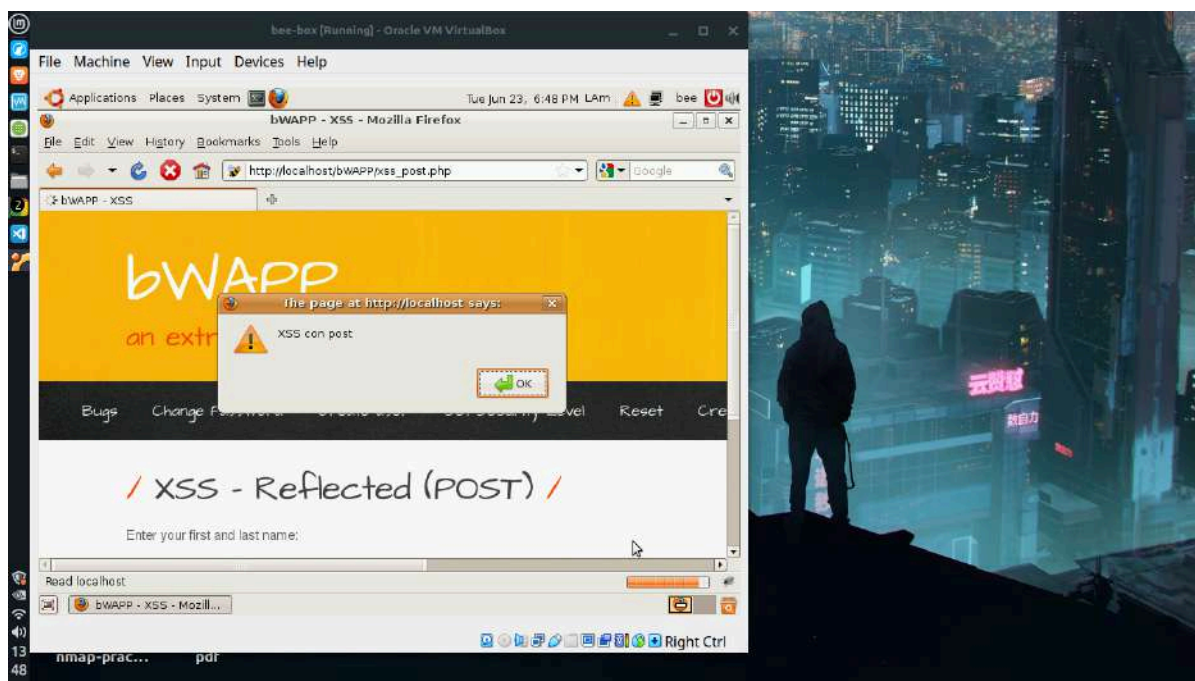
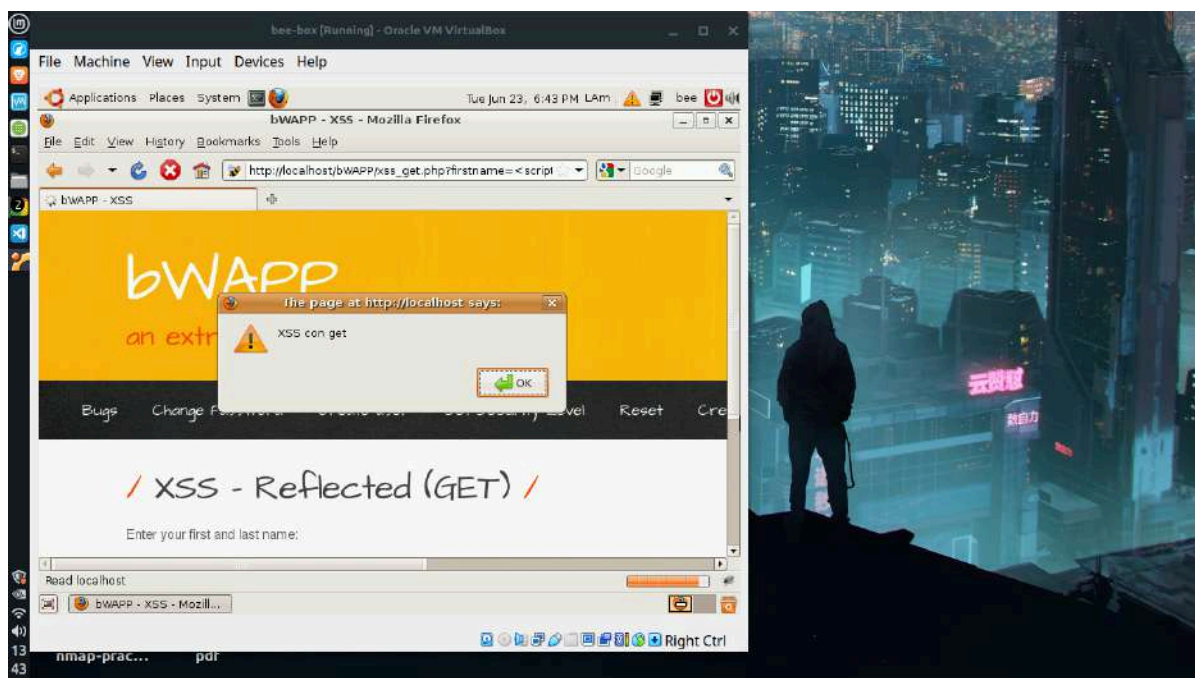


4. Injection — Cross-Site Scripting (XSS) Reflected (GET y POST)

Categoría OWASP: A03 Injection.

El ejercicio demuestra XSS reflejado en sus dos variantes. La aplicación devuelve en el HTML el contenido ingresado en el formulario sin sanitizarlo, por lo que un script enviado como entrada se ejecuta en el navegador. En la variante GET, el payload viaja visible en la URL y se confirmó la ejecución mediante una ventana de alerta. En la variante POST, el mismo ataque se ejecuta igual, pero los datos viajan en el cuerpo de la petición y la URL permanece limpia, lo que hace el ataque más sigiloso aunque más difícil de distribuir. En ambos casos la causa raíz es la falta de escapado del input antes de reflejarlo.

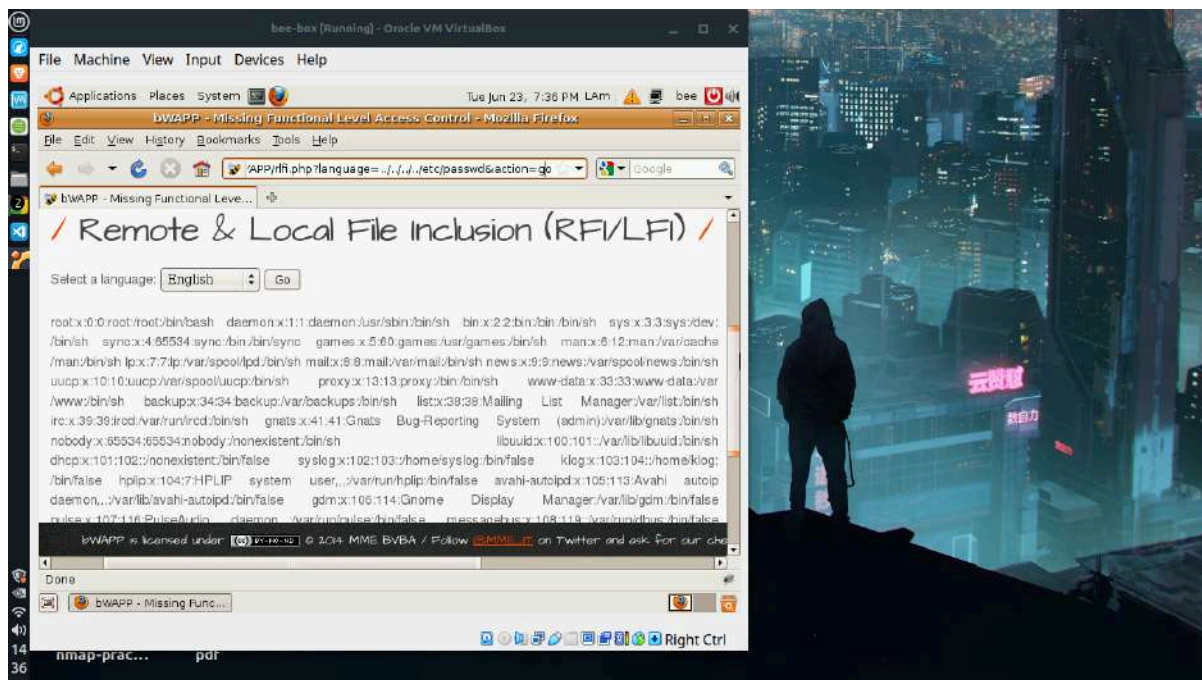


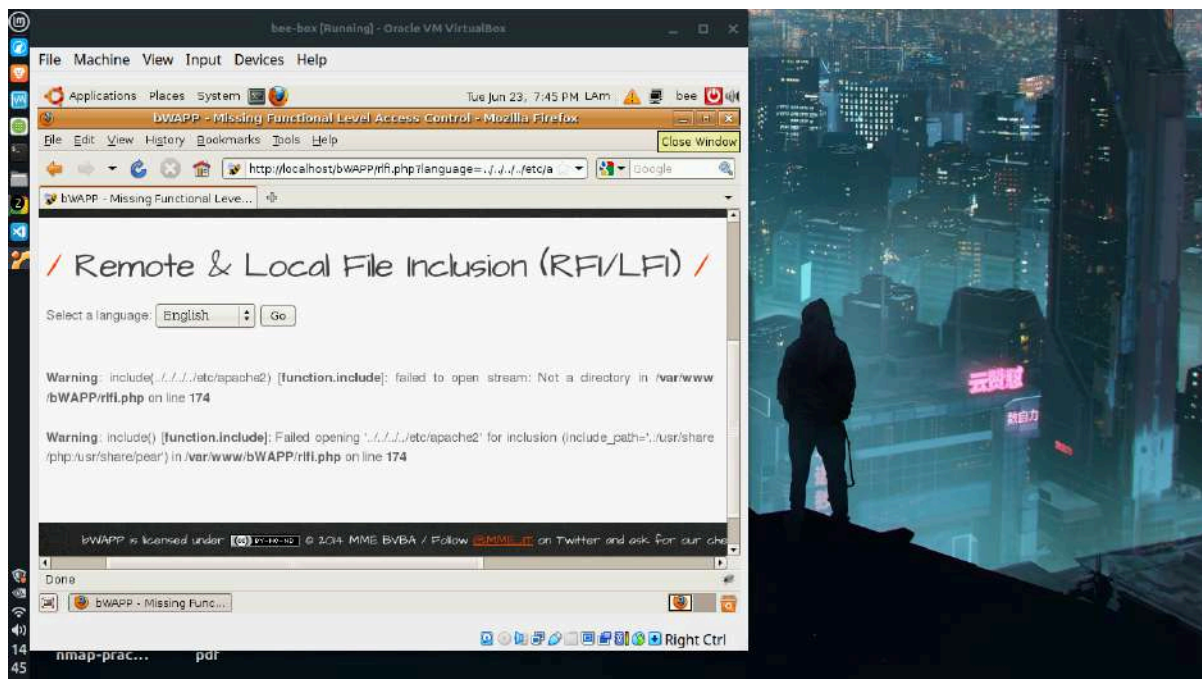
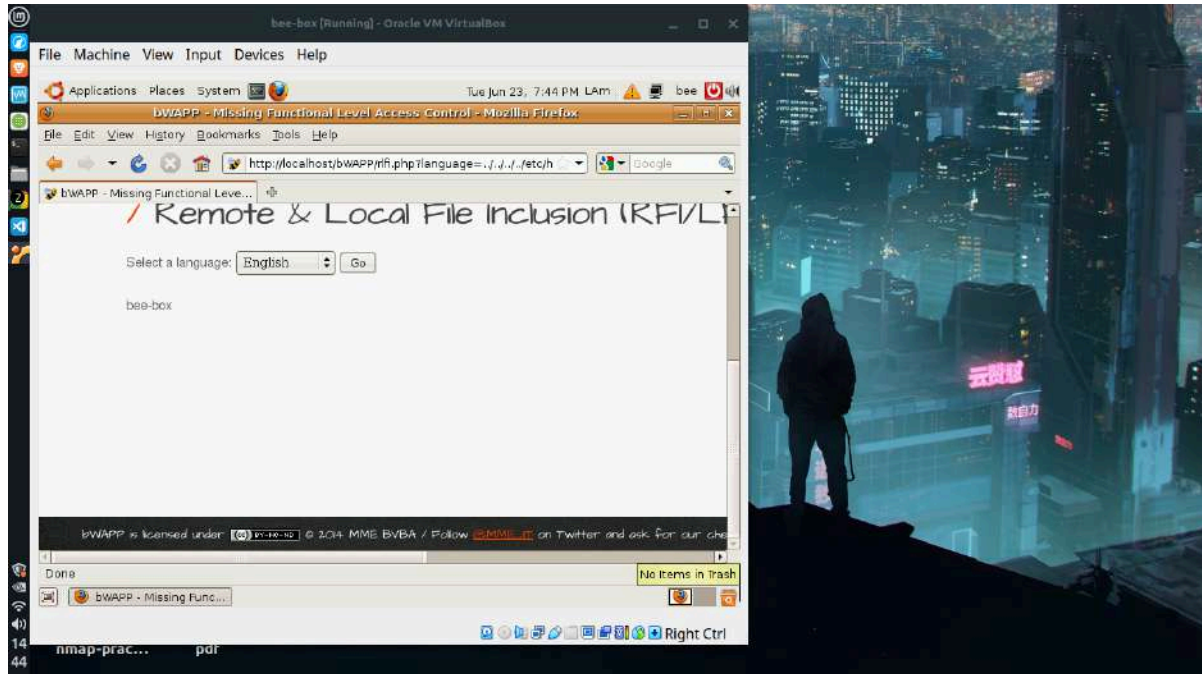


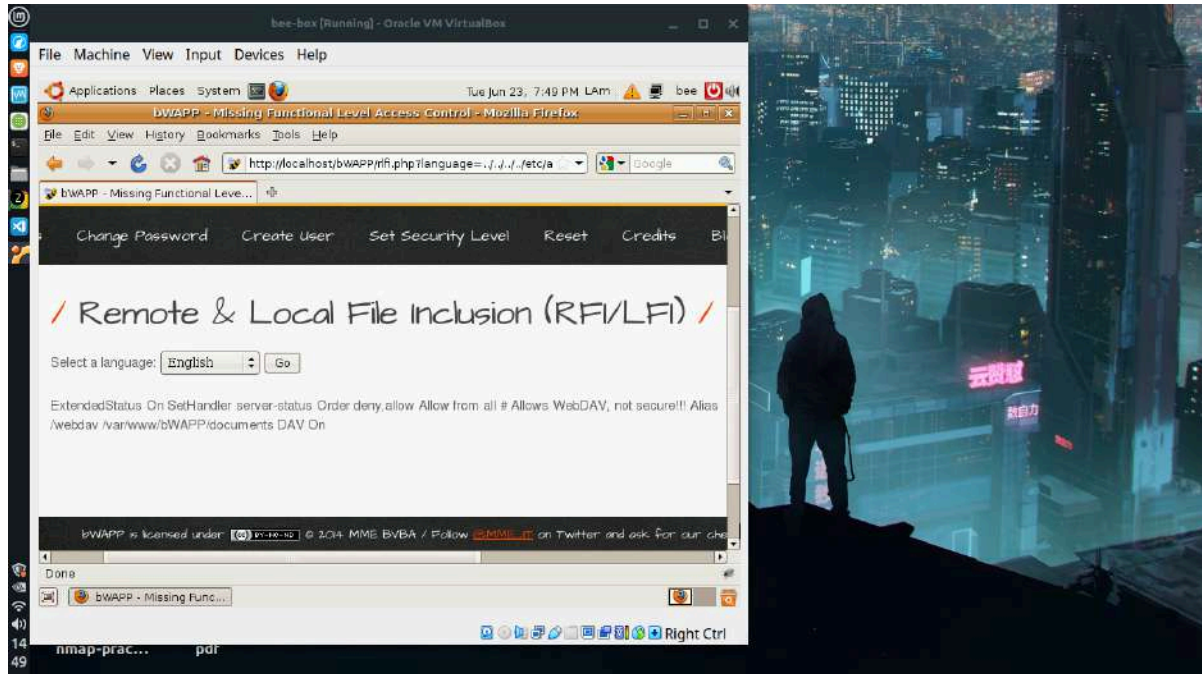
5. Security Misconfiguration — Local File Inclusion (LFI)

Categoría OWASP: A05 Security Misconfiguration.

El ejercicio explota una inclusión de archivos locales a través del parámetro language, que la aplicación utiliza para cargar archivos de idioma sin validar la ruta recibida. Mediante una secuencia de path traversal se accedió al archivo `/etc/passwd` del servidor, revelando los usuarios del sistema. Se confirmó además la lectura del hostname (`bee-box`) y de la configuración de Apache. Durante la exploración, los mensajes de error de PHP divulgaron información sensible, como la ruta absoluta de la aplicación y la línea exacta del código vulnerable. La lectura del archivo de configuración del servidor expuso, adicionalmente, una configuración insegura de WebDAV reconocida en el propio comentario del archivo.



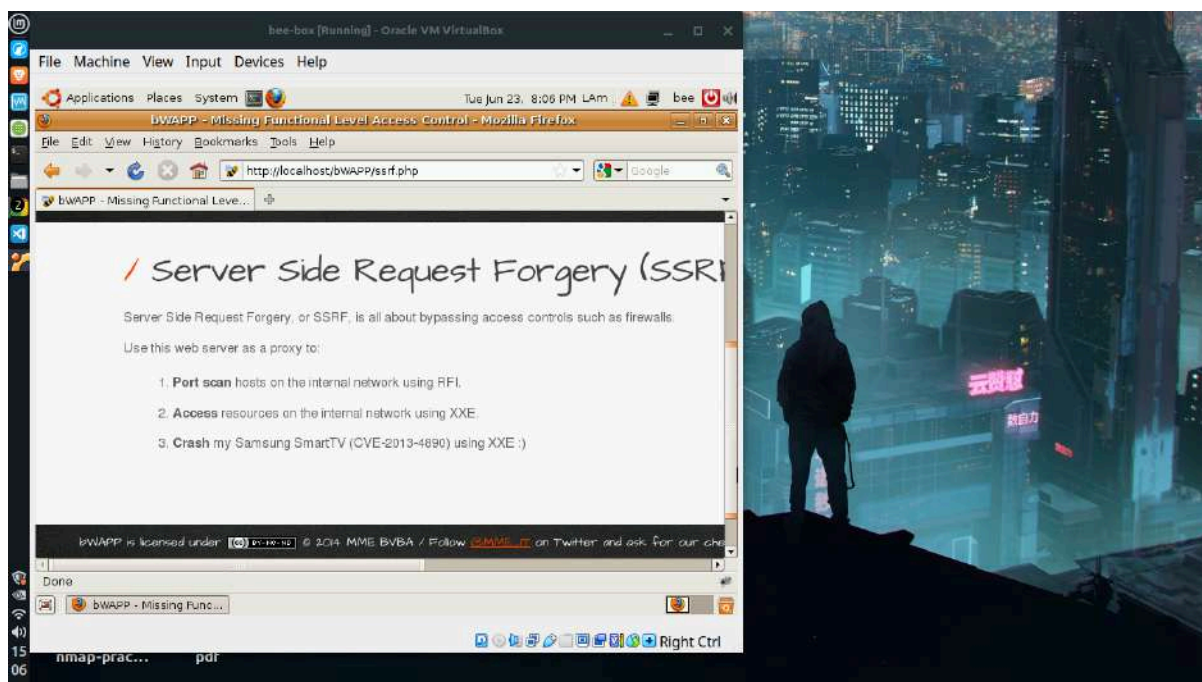


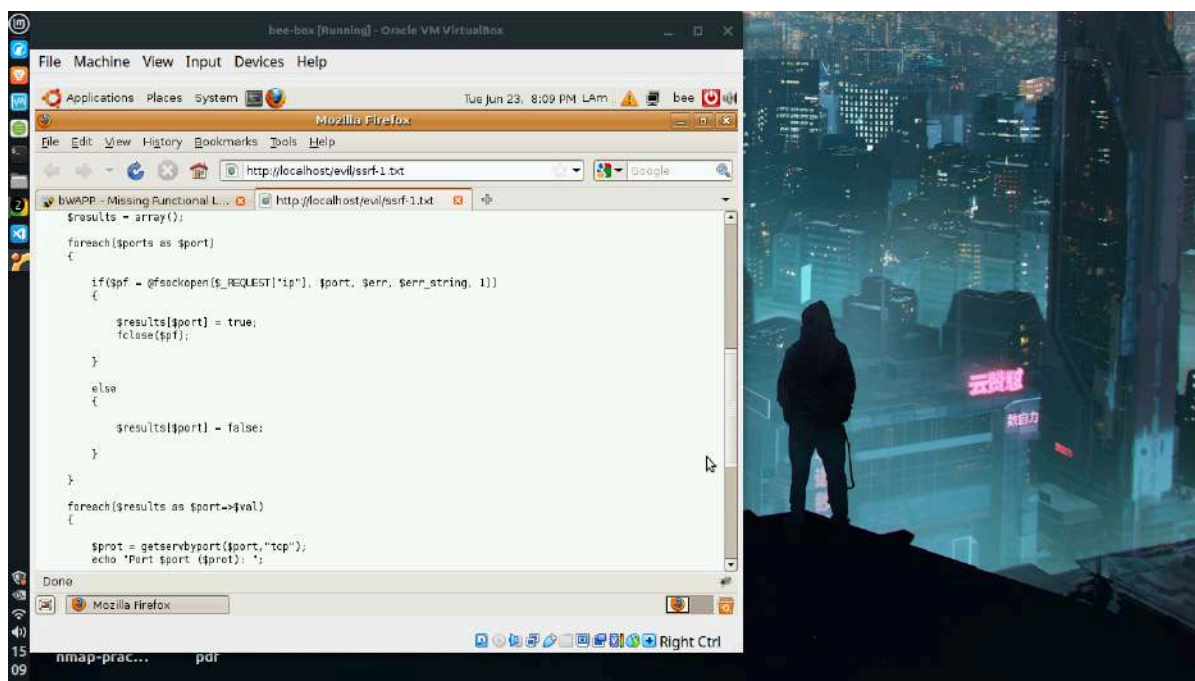
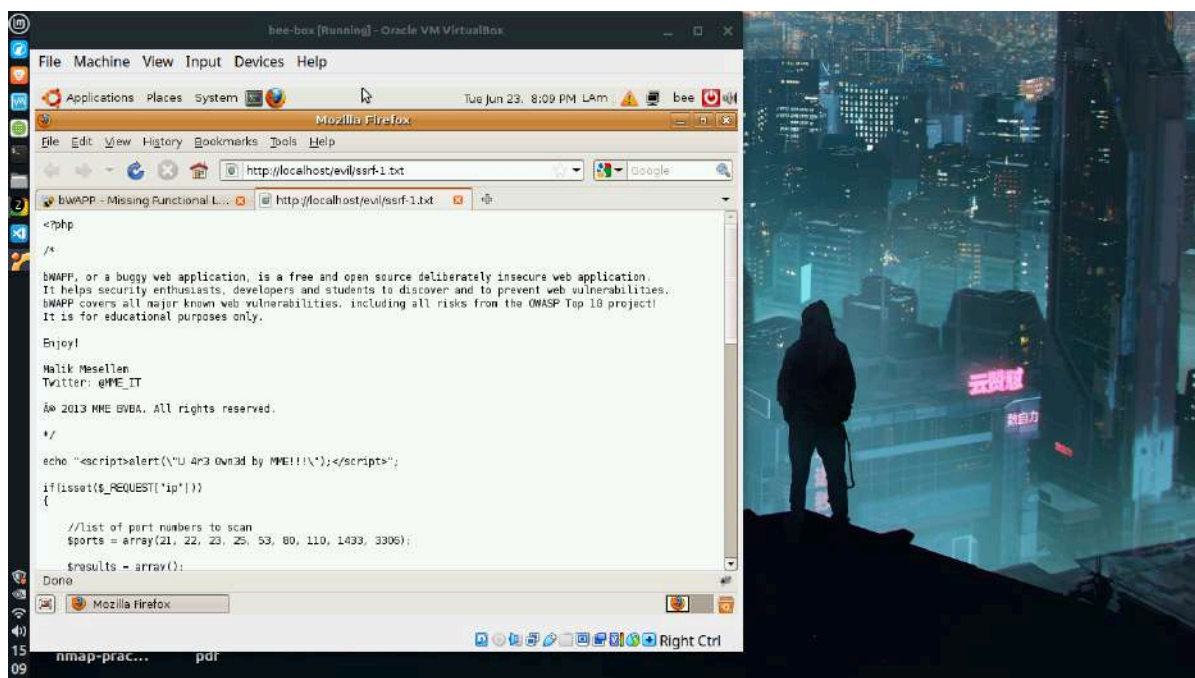


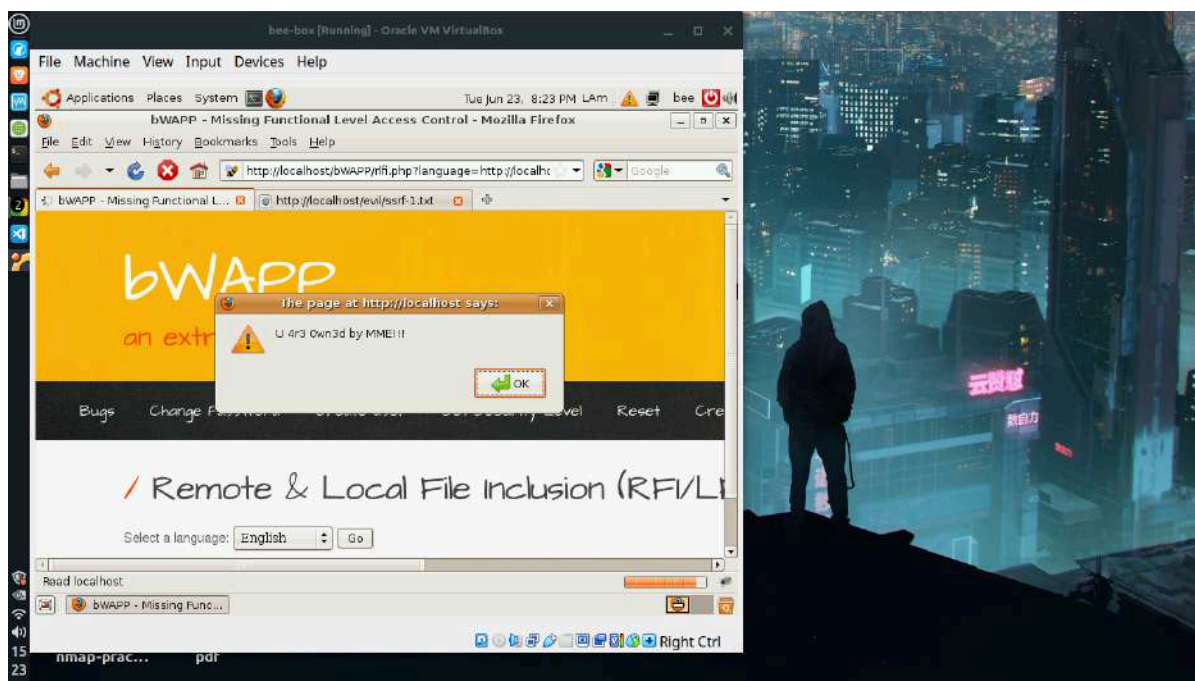
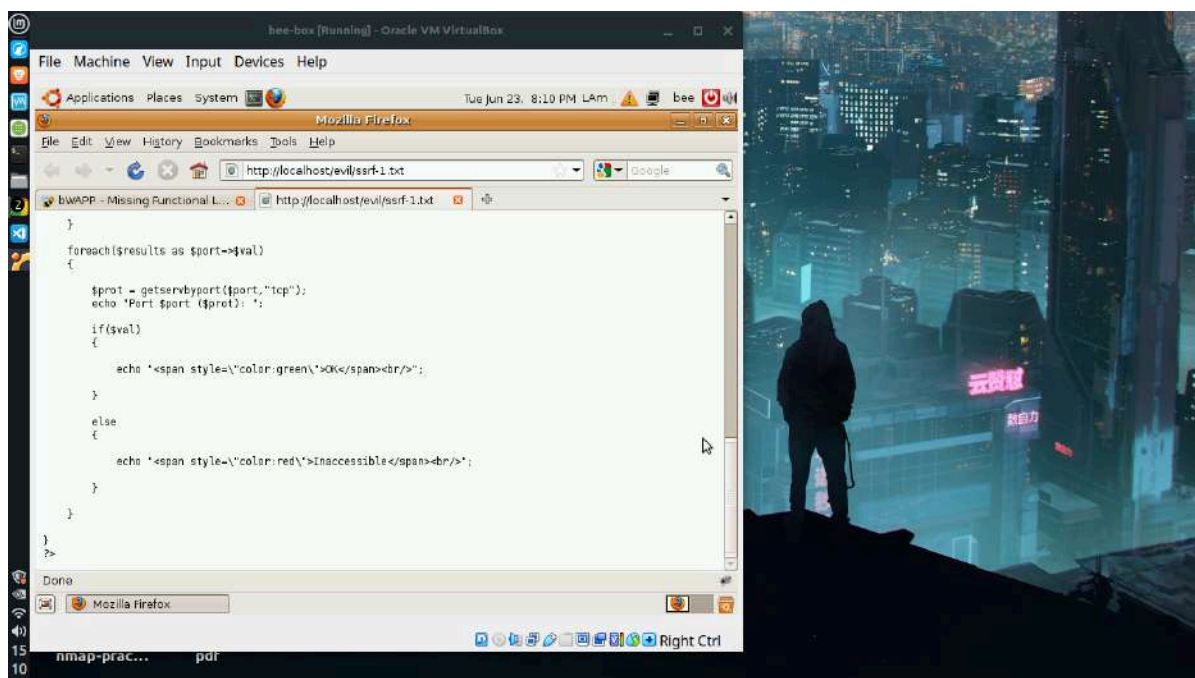
6. Server-Side Request Forgery (SSRF) — Port Scanning interno

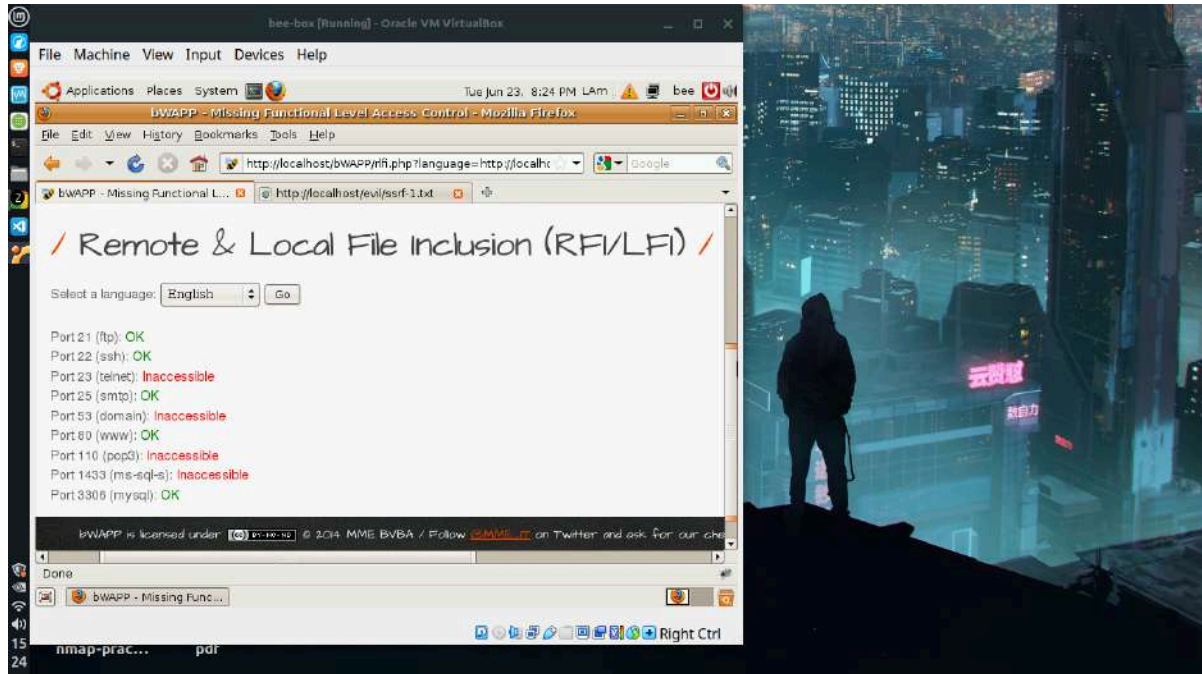
Categoría OWASP: A10 Server-Side Request Forgery.

El ejercicio combina la inclusión remota de archivos (RFI) con SSRF para realizar un escaneo de puertos de la red interna usando el servidor como proxy. A través del parámetro language se incluyó un archivo remoto que contiene un script PHP de escaneo de puertos. El script, ejecutado desde el propio servidor, prueba la conexión a una lista de puertos comunes sobre la dirección indicada. El resultado mostró los puertos abiertos del servidor (FTP, SSH, SMTP, HTTP y MySQL) y los cerrados. La importancia del ataque radica en que el servidor opera dentro de la red interna, por lo que puede alcanzar recursos inaccesibles desde el exterior, evadiendo así los controles de acceso como firewalls.





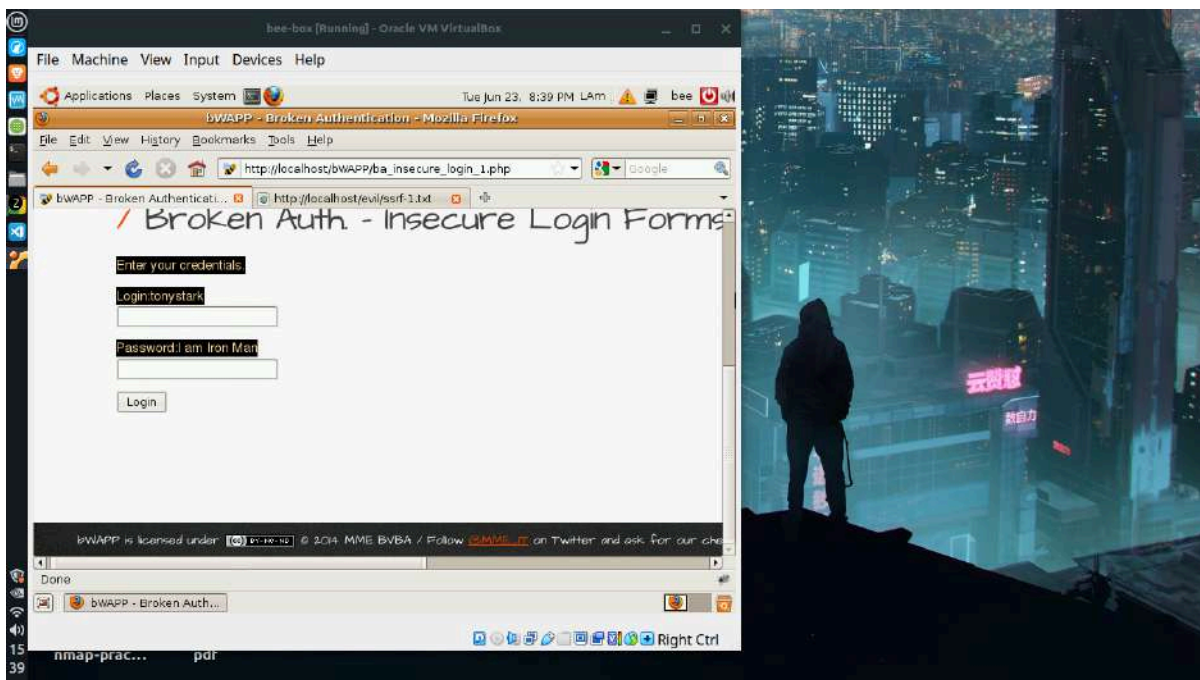


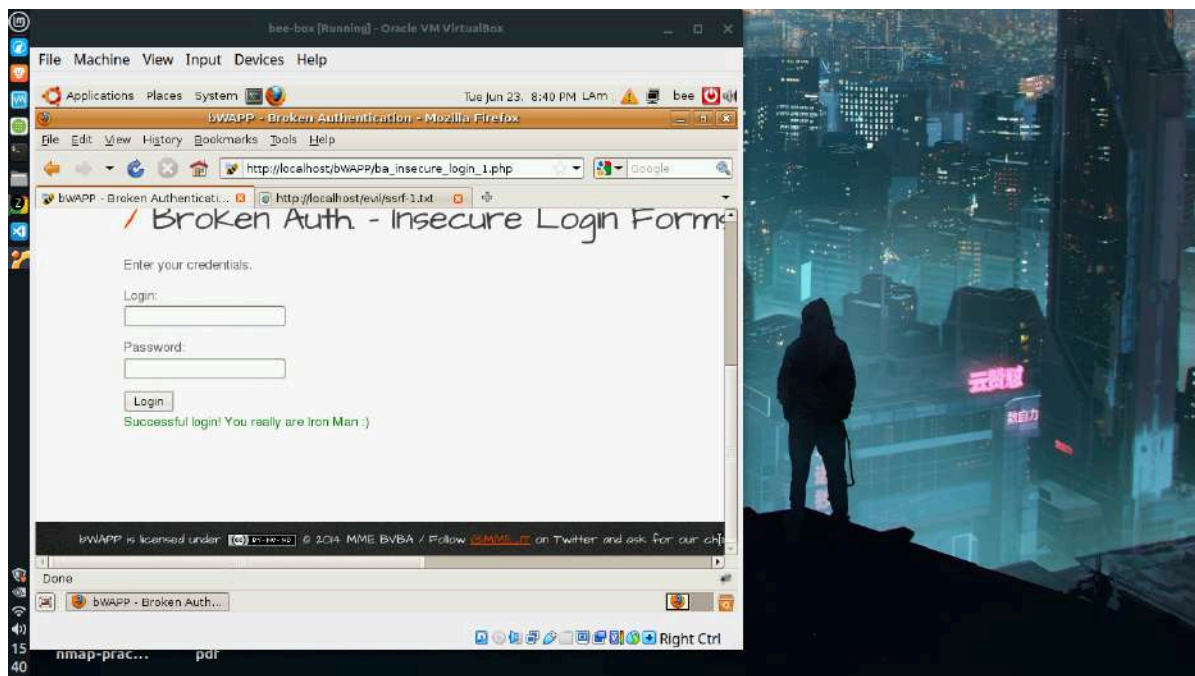


7. Insecure Design — Login Page

Categoría OWASP: A04 Insecure Design.

El ejercicio ilustra una falla de diseño inseguro a través de la página de login. Las credenciales del usuario `tonystark` están presentes en el formulario pero ocultas visualmente, pintadas del mismo color que el fondo. Seleccionando el texto con el mouse o inspeccionando el elemento, las credenciales se vuelven visibles y permiten acceder al sistema. A diferencia de un error de implementación, este es un problema de concepción: alguien diseñó el sistema asumiendo que ocultar visualmente un dato equivale a protegerlo, una forma de seguridad por oscuridad. Este tipo de falla no se corrige con un parche, sino que requiere rediseñar el mecanismo de autenticación desde su base.

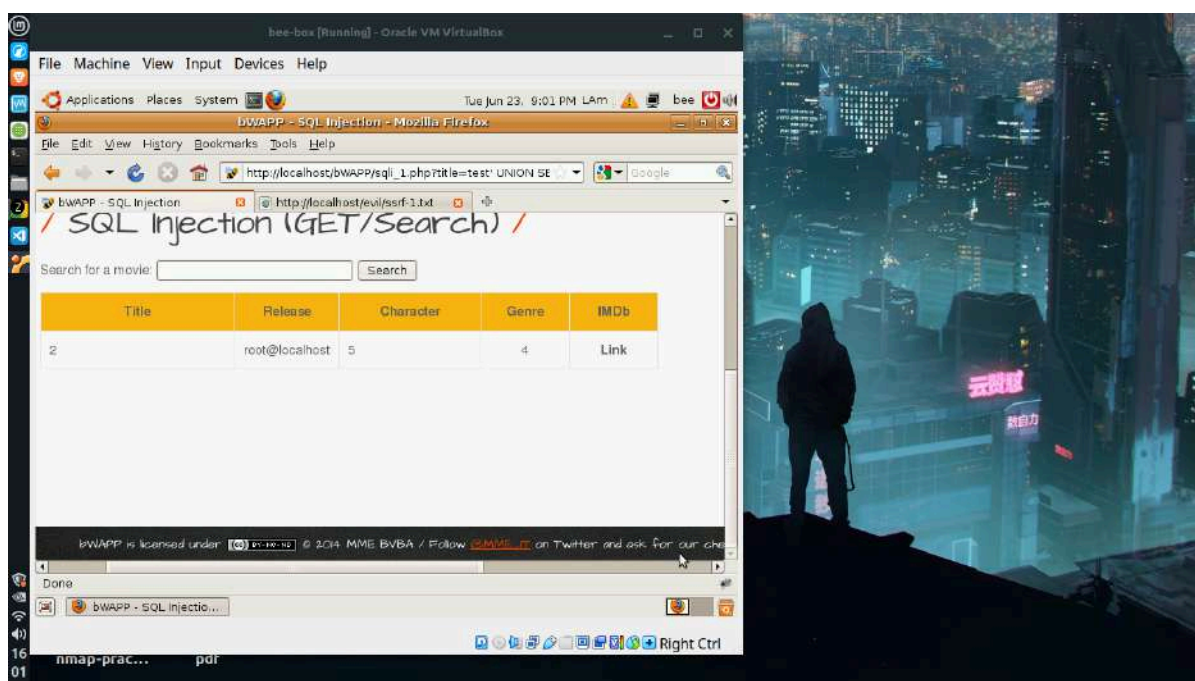


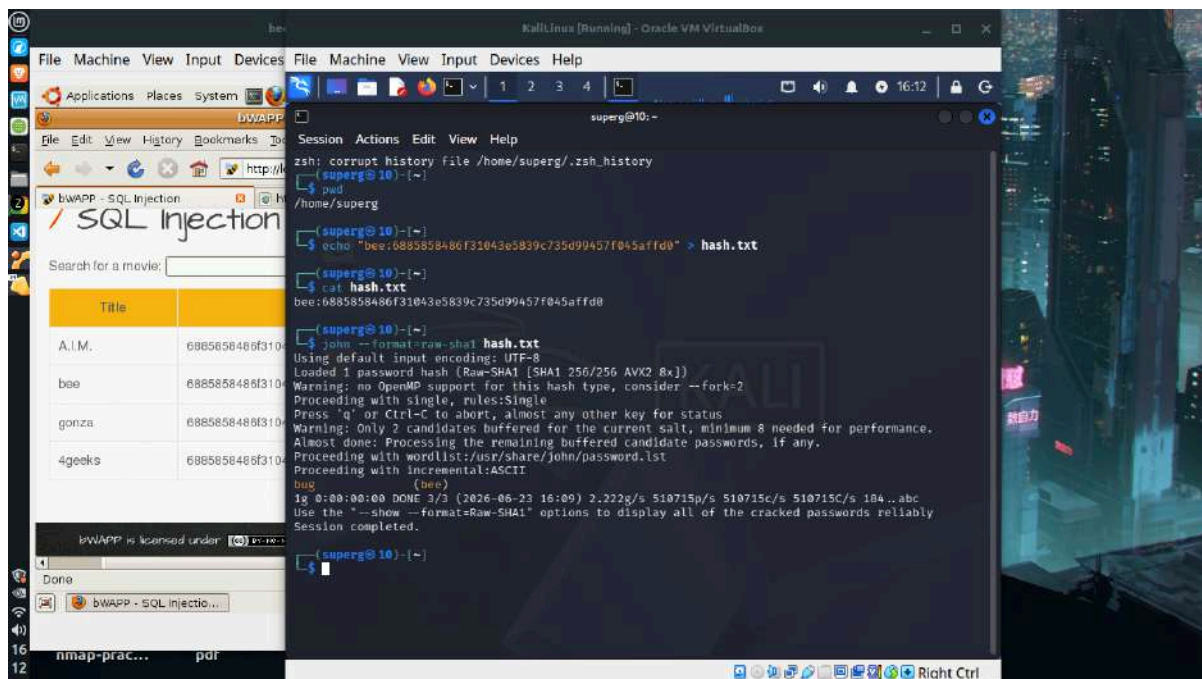
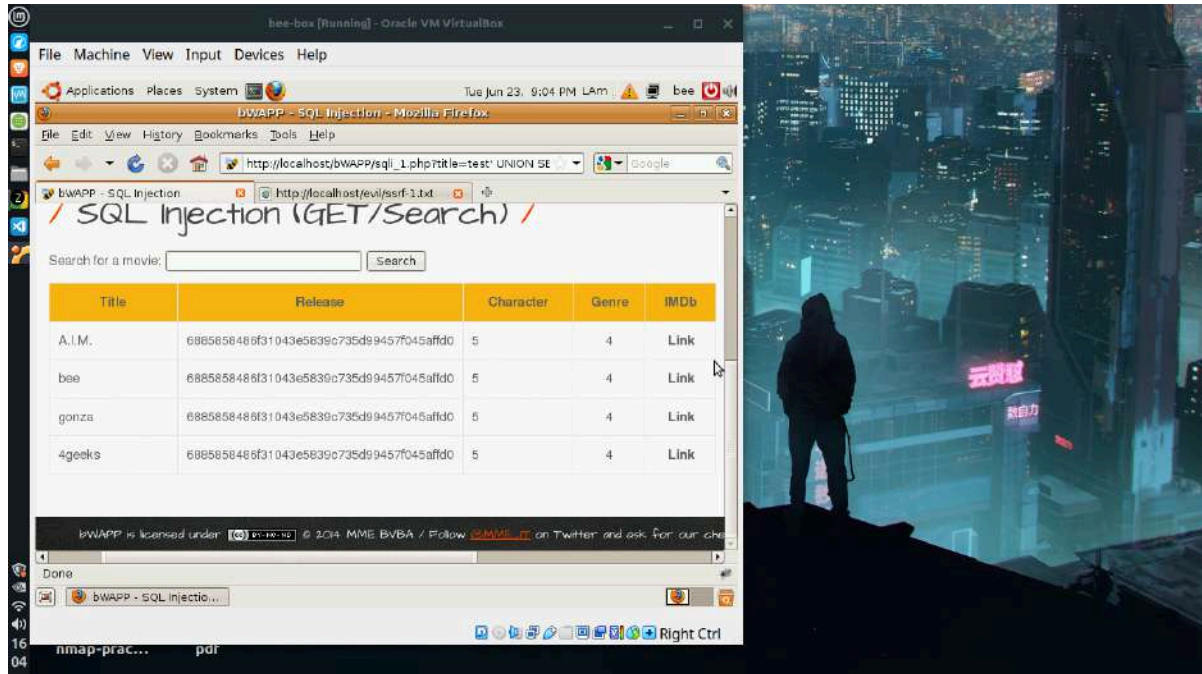


8. Cryptographic Failures — Weak Password Hashing

Categoría OWASP: A02 Cryptographic Failures.

El ejercicio encadena una inyección SQL con el cracking de hashes para demostrar el uso de un algoritmo criptográfico débil. Mediante SQL injection se extrajeron los hashes de contraseña de todos los usuarios, observándose que los cuatro usuarios comparten el mismo hash SHA-1, consecuencia de usar la misma contraseña sin aplicar salt. El hash se guardó en un archivo y se procesó con John the Ripper, que recuperó la contraseña en texto plano (bug) en menos de un segundo. Finalmente se verificó el acceso a la aplicación con la credencial recuperada. SHA-1 resulta inadecuado para contraseñas por su velocidad de cálculo y la ausencia de salt, lo que lo hace vulnerable a ataques de diccionario y tablas precalculadas.

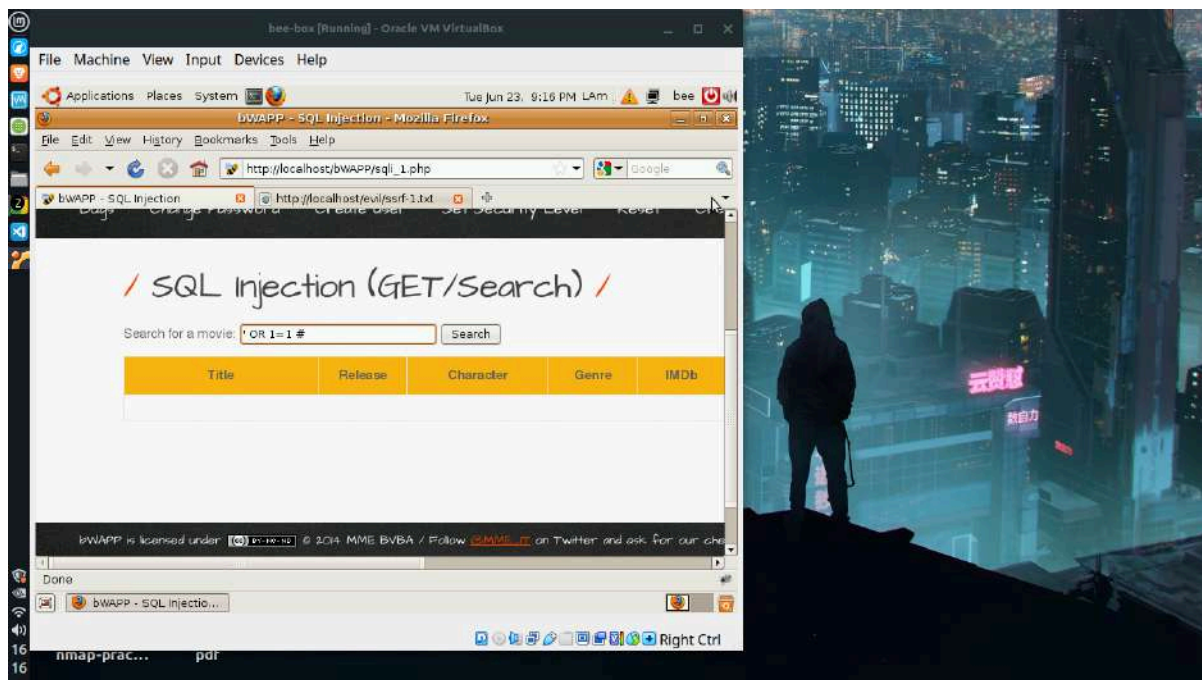


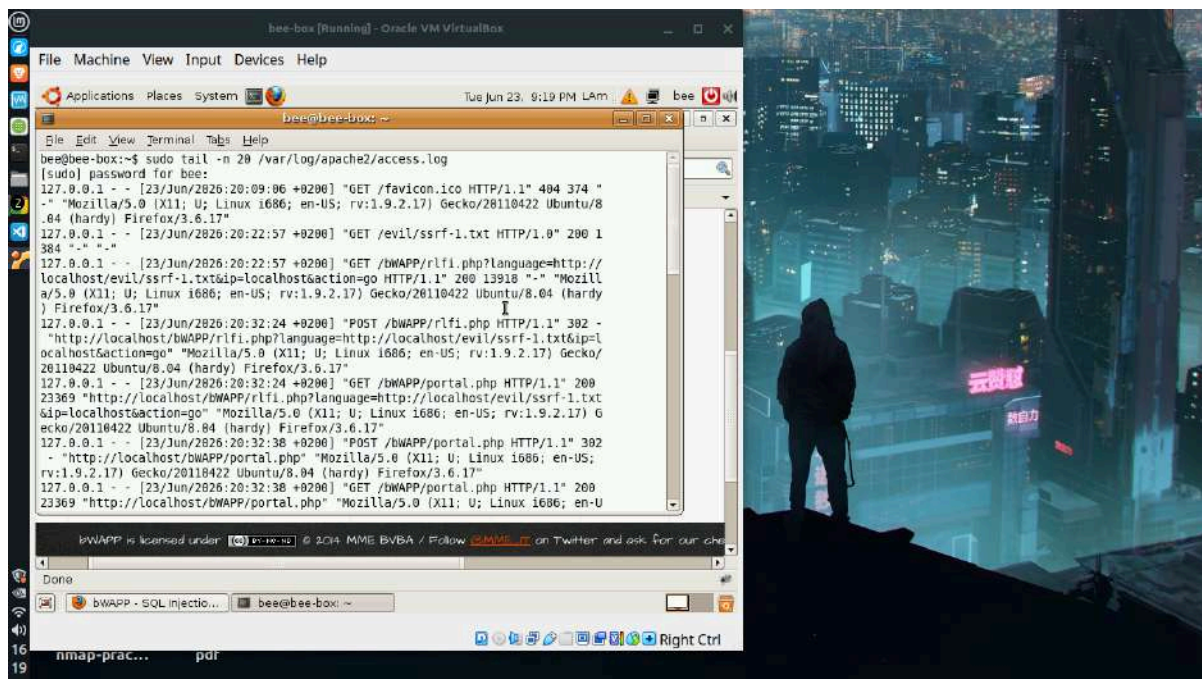
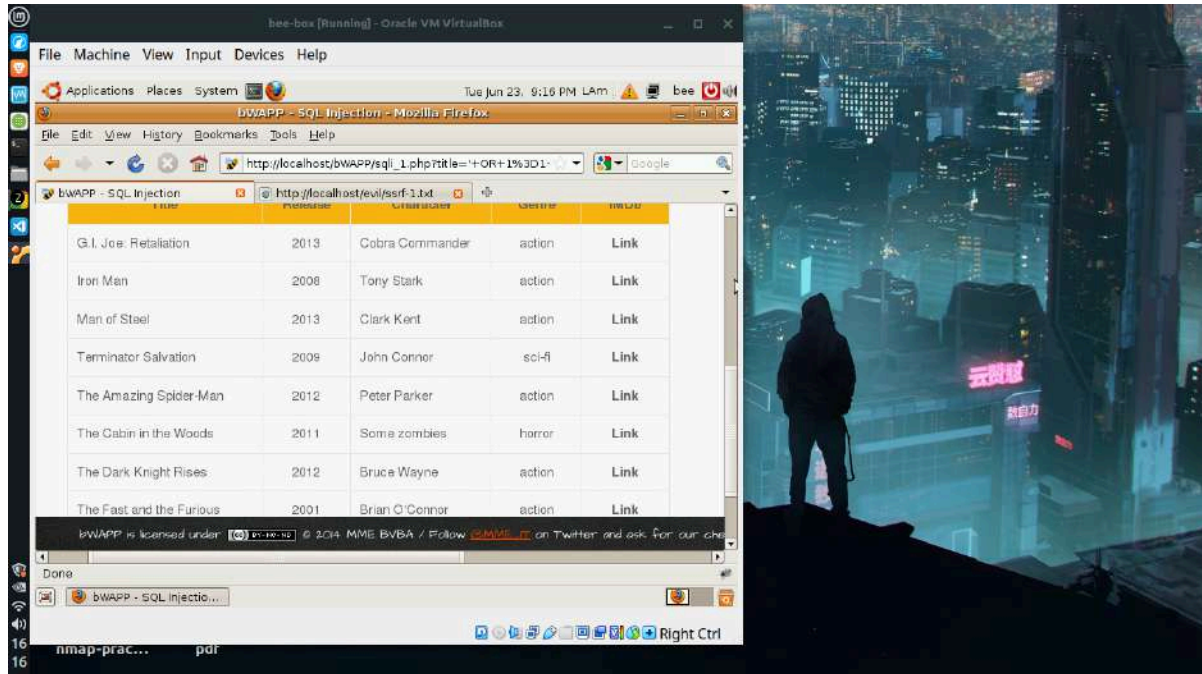


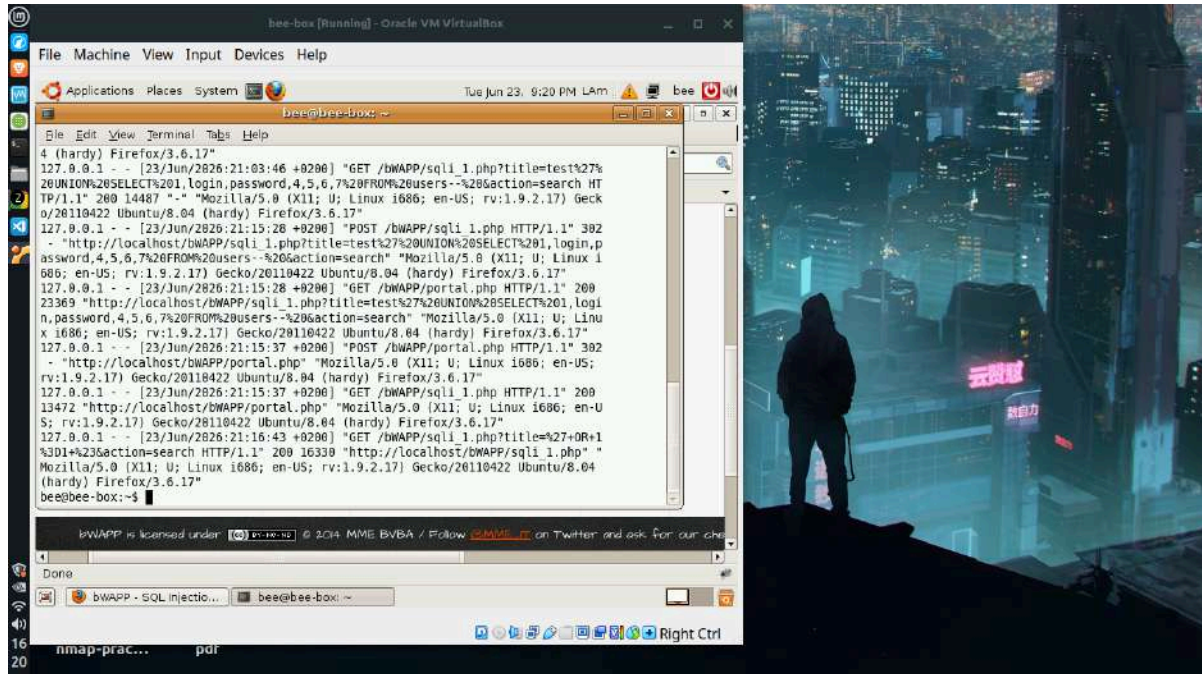
9. Security Logging and Monitoring Failures

Categoría OWASP: A09 Security Logging and Monitoring Failures.

El ejercicio adopta la perspectiva defensiva para evaluar si la aplicación detecta los ataques. Se ejecutó una inyección SQL simple con el payload de condición siempre verdadera, que devolvió la tabla completa de películas, confirmando la vulnerabilidad. A continuación se revisaron los registros de acceso de Apache, donde quedó constancia del ataque con su fecha, dirección de origen y el payload completo. Sin embargo, la petición maliciosa fue registrada con código de respuesta 200 (exitoso), igual que una búsqueda legítima, sin generar ninguna alerta ni bloqueo. Esto evidencia la diferencia entre registrar y monitorear: el sistema deja constancia de los eventos pero no reacciona ante ellos, lo que impide la detección y respuesta en tiempo real.







10. Vulnerable and Outdated Components — Stored XSS (Blog)

Categoría OWASP: A06 Vulnerable and Outdated Components.

El ejercicio demuestra un XSS almacenado (persistente) en el módulo de blog para ilustrar el riesgo de componentes desactualizados. A diferencia del XSS reflejado, el script malicioso ingresado como comentario se guarda en la base de datos y se ejecuta cada vez que la página se carga, afectando a todos los usuarios que la visiten, incluido el administrador. Se confirmó que el payload quedó almacenado y se dispara automáticamente en cada recarga. La causa raíz es la ausencia de validación y escapado del input antes de almacenarlo y mostrarlo, una práctica que componentes y librerías modernas mitigan de forma automática. Es la variante más peligrosa de XSS por su carácter persistente y de alcance masivo.

